Vitto Lending Decision System — Architecture Write-up

Overview

This project is a lightweight lending decision system for MSME loan applications. It collects a business profile and loan request, evaluates the request using an explainable scoring model, and returns a structured decision with a credit score and reason codes.

Stack: React (SPA) · Node.js/Express (REST API) · PostgreSQL (system of record + audit trail)

High-level Architecture

```
React SPA (Port 5173)
  -> Node/Express API (Port 4000/4001)
    -> Validation middleware (Zod transport validation)
    -> Use cases (application orchestration)
      -> Decision engine (pure business logic)
      -> Repositories (Postgres persistence)
      -> Audit logger (Postgres persistence)
```

Key design choice: keep the decision logic isolated so it is testable, explainable, and easy to defend.

Data Storage Strategy (PostgreSQL)

PostgreSQL is the system of record for:

- Business profiles
- Loan applications
- Credit decisions (including reason codes and derived metrics snapshot)
- Audit events (append-only trail of profile creation, application creation, and decision creation)

Why use PostgreSQL for both:

- The core domain data is relational and benefits from constraints and joins
- Audit events are simple structured records that fit well in a relational table
- Single database simplifies deployment, backup, and operations
- No need to manage a second database connection or dependency

Audit failure behavior:

- If the audit insert fails, the API still returns success (the core decision is already persisted); the server logs the audit failure for visibility.

Decision Engine Design

The decision engine is implemented as a pure domain module:

Hard boundaries (non-responsibilities):

- Does not know about Express
- Does not know about SQL
- Does not write logs directly
- Does not fetch data itself

It accepts already-loaded inputs (business profile + loan application) and returns:

- `APPROVED` or `REJECTED`
- A computed credit score (0–100)
- One or more reason codes
- Derived metrics used to explain the decision

Why this design:

- Deterministic, unit-testable logic without needing a running server or database
- Prevents business logic from being scattered inside HTTP handlers or persistence code
- Makes it easier to defend thresholds and outcomes in documentation and interview discussion

---

Scoring Model (Explainable and Documented)

The scoring model prioritizes explainability over complexity. Signals include:

- Revenue-to-EMI ratio (monthly revenue vs estimated monthly repayment)
- Loan-to-revenue multiple

- Tenure risk (very short or very long tenures)
- Basic consistency checks (loan amount disproportionate to revenue)

The model:

- Starts with a base score of 100
- Applies penalties per risk signal
- Can hard-reject on invalid or extreme inconsistencies

Reason codes:

- `LOW_REPAYMENT_CAPACITY`
- `HIGH_LOAN_RATIO`
- `EXTREME_TENURE`
- `DATA_INCONSISTENCY`
- `INVALID_PAN_FORMAT`

All thresholds and assumptions are documented in README.md and docs/API.md.

Edge Case Handling

Edge case handling is implemented at both the frontend and backend boundaries:

- **Missing fields:** client-side required field checks and server-side Zod validation
- **Invalid formats:** structured validation errors (e.g., malformed PAN, negative numbers, non-numeric values)

- **Conflicting data:** valid request is accepted but decision is rejected with explainable reason codes
- **Database failures:** graceful error responses; audit failure does not block core decision
- **Rate limiting:** 429 responses on excessive decision requests

Goal: no unhandled exceptions in normal flows; error responses are consistent and structured.

API Design Decisions

Consistent Envelope

Every response uses the same envelope shape:

```
{ "data": { ... }, "meta": { "requestId": "uuid" } }
```

Errors use the same envelope with an `error` key instead of `data`.

Currency Handling

- API accepts and returns rupees (integers)
- Controllers convert rupees to paise before passing to domain layer
- PostgreSQL stores paise as BIGINT to avoid floating-point errors
- Domain layer works with paise (Money value object)

Request IDs

Every request gets a UUID attached to `res.locals.requestId`:

- Included in every response envelope
- Passed to audit logs for traceability
- Helps debug issues across logs

Tradeoffs Chosen

- **Synchronous decisioning:** chosen for simplicity and reliability in a time-boxed sprint; async job queue was intentionally deferred (bonus only).
- **Format-only PAN validation:** chosen because real verification would require external integrations outside assignment scope.
- **Minimal UI:** prioritized clarity and correctness over complex UI states.
- **Paise storage in Postgres:** avoids floating-point rounding issues by storing money as integers.
- **Node.js built-in test runner:** avoids external test framework dependencies; works out of the box with Node 20+.

Improvements With More Time

- **Async decision processing** (job + polling endpoint)

- **More robust audit/event taxonomy and dashboards**
- **Decision engine calibration** using sample datasets and more nuanced risk weights
- **Stronger observability:** structured logs (Pino), metrics (Prometheus), tracing
- **Stronger API hardening:** stricter rate limiting, abuse detection, request size constraints
- **Admin dashboard** to view audit trail and decision trends
- **Frontend testing** with React Testing Library
- **CI/CD pipeline** with GitHub Actions